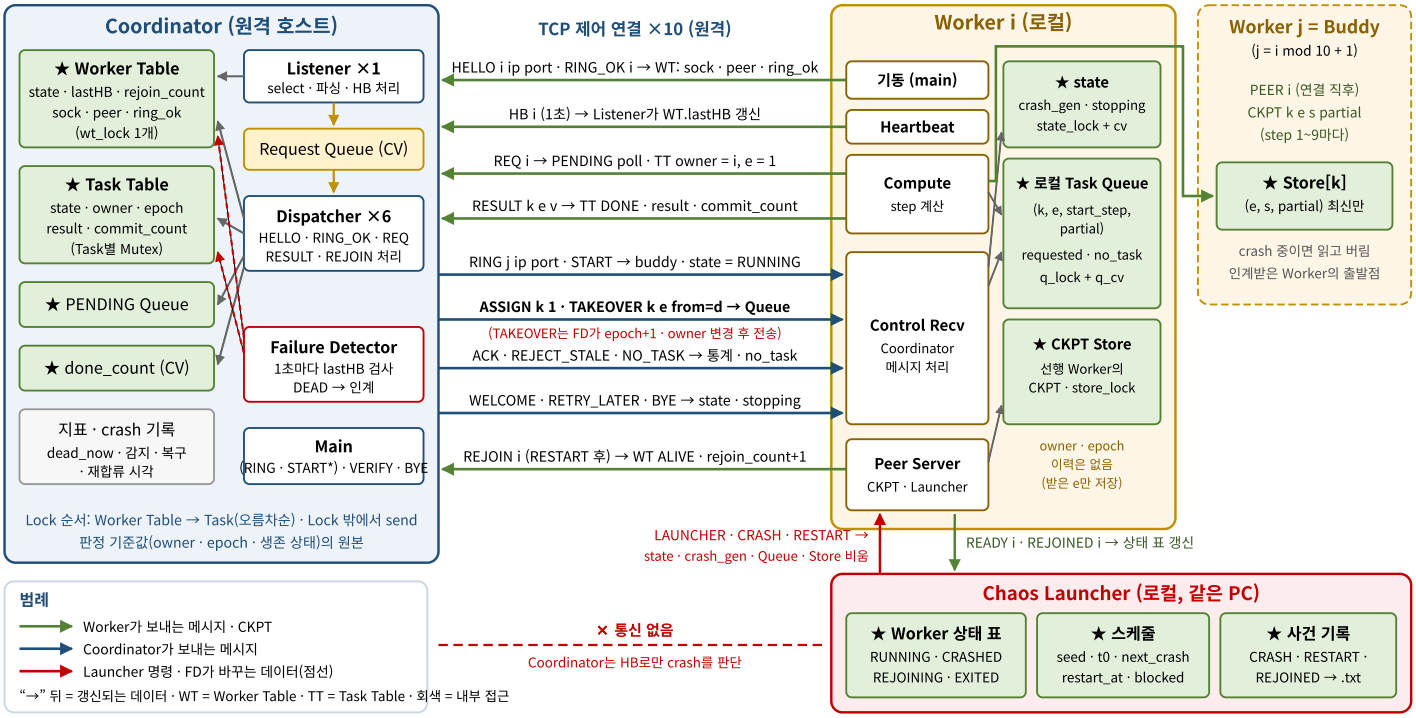
HW#3 구현 데이터 설계 — 컴포넌트별 유지·관리 데이터와 제안 양식

★ = 명세상 반드시 있어야 하는 데이터 · 필드 이름과 자료구조는 자유(제안안) · 근거: 과제설명 PPTX p.10 · 12 · 14 · 15 · 24 · 33 · 37 · 38–39 · 41–42 · 54

0. 컴포넌트 관계와 송수신 데이터 흐름



메시지별 송수신과 읽고 바꾸는 데이터

메시지	보내는 쪽 → 받는 쪽 (Thread)	받는 쪽이 읽는 데이터	받는 쪽이 바꾸는 데이터
HELLO i ip port	Worker → Listener → Dispatcher	—	WT: sock · peer · sock_to_id (10개 모이면 RING 전송)
RING j ip port	10번째 HELLO를 처리한 Dispatcher 또는 Main* → Control Recv	WT.peer (보내는 쪽)	Worker: buddy 주소 → Buddy에 연결 후 `PEER i`
RING_OK i / START	Worker → Dispatcher / 10번째 RING_OK를 처리한 Dispatcher 또는 Main* → Control Recv	—	WT.ring_ok (10개 → START) / Worker state = RUNNING, Coordinator lastHB 초기값
HB i	Heartbeat → Listener	WT.state	WT.lastHB (SUSPECT → ALIVE 복귀, DEAD면 무시)
REQ i	Compute → Dispatcher (보내기 전 Worker: requested = True)	WT.state (DEAD면 무시)	PENDING poll → TT: ASSIGNED · owner = i · epoch = 1
ASSIGN k 1 / NO_TASK	Dispatcher → Control Recv	—	Queue에 (k, 1, 1, 0), requested = False / no_task = True
CKPT k e s partial	Compute → Buddy의 Peer Server	Store[k]	Buddy의 Store[k] ((e, s)가 더 크면 덮어씀, crash 중이면 버림)
RESULT k e v	Compute → Dispatcher	WT.state · TT[k] (epoch · owner)	TT: DONE · result · commit_count+1 → done_count+1 / 아니면 REJECT_STALE
ACK / REJECT_STALE	Dispatcher → Control Recv	—	통계 · 로그만 (모르는 Task면 무시)
(FD 판정)	Failure Detector 내부	WT.lastHB · TT.owner	WT.state (SUSPECT · DEAD), TT[k]: epoch+1 · owner = b
TAKEOVER k e from=d	FD → b의 Control Recv	b의 Store[k]	b의 Queue에 (k, e, 저장 step+1, 저장 partial) 또는 (k, e, 1, 0)
LAUNCHER	Launcher → Peer Server (연결 직후 첫 메시지)	—	그 연결을 Launcher 제어 연결로 기록, launcher_connected = True (READY 조건)
CRASH	Launcher → Peer Server	—	state = CRASHED · crash_gen+1 · Queue · Store · 플래그 삭제 / Launcher: status = CRASHED
RESTART → REJOIN i	Launcher → Peer Server → (REJOIN) Dispatcher	WT.state	Worker state = REJOINING / WT: ALIVE · lastHB · rejoin_count+1
WELCOME / RETRY_LATER	Dispatcher → Control Recv	state	state = RUNNING (notify_all) / 2초 후, 아직 REJOINING이면 REJOIN 재전송
READY i / REJOINED i	Worker → Launcher	—	Launcher: ready_count+1 (10개 → t0) / status = RUNNING
BYE	Main → Control Recv	—	stopping = True → Worker 종료 → Launcher: 제어 연결 EOF → EXITED

* RING · START를 보내는 Thread는 조 자유(PPTX p.10의 Main "Ring 구성 · START 조율" 또는 마지막 HELLO · RING_OK를 처리한 Dispatcher). 선택한 방식을 Readme에 적는다. 어느 쪽이든 START 시각을 모든 lastHB 초기값으로 두고 FD를 시작한다.

1. Coordinator

★ Task Table [0..999] — Task마다 Mutex 1개

필드	형식	초기값	바꾸는 Thread
k	int 0~999	0..999	—
state	PENDING / ASSIGNED / DONE	PENDING	Dispatcher(REQ · RESULT)
owner	Worker 번호 / None	None	Dispatcher(ASSIGN), FD(인계)
epoch	int	0	ASSIGN = 1, 인계마다 +1
result	int / None	None	Dispatcher(Commit)
commit_count	int	0	Dispatcher(Commit), 종료 시 모두 1
lock	Mutex	—	Task Table 전체 Lock 1개 금지

★ Worker Table [1..10] — 전체에 Lock 1개 (wt_lock)

필드	형식	설명
state	ALIVE / SUSPECT / DEAD	FD가 HB로 판정
lastHB	시각	Listener 갱신, 초기값 = START 시각
rejoin_count	int	REJOIN 승인마다 +1
sock	Socket	제어 연결 (HELLO 때 등록)
send_lock	Mutex	Socket마다 송신 Lock
peer_ip, peer_port	str, int	HELLO로 받음 → RING에 사용
ring_ok	bool	10개 모두 True → START

그 밖의 공유 데이터

이름	형식	보호	용도
★ sock_to_id	dict Socket → 번호	wt_lock	보낸 Worker 판별 (HELLO 처리 때 등록)
★ PENDING Queue	deque 0..999 오름차순	wt_lock (REQ 구간 안에서 poll — 별도 Lock 불필요, thread-safe Queue도 가능)	다음 배정 번호
★ Request Queue	deque (sock, msg) — HELLO는 번호 등록 전이므로 sock으로 넣고 Dispatcher가 sock_to_id로 번호 확인	Mutex + CV(not_empty)	Listener → Dispatcher 6, 종료 시 STOP×6
★ done_count	int + CV(all_done)	종료 Mutex	1,000번째 Commit → Main
recv_buf[sock]	bytes	Listener 전용	`\n` 단위 파싱
stopping	bool	—	VERIFY 후 FD · Listener 종료
total (선택)	int	Mutex	방법 B: Commit마다 누적

지표 · crash 기록 (crash마다 1행 → Readme 표)

필드	예
no, worker	1, W4
lastHB, dead_time	14.95, 20.00 (감지 지연 5.05초)
moved_tasks → to	[152] → W5
recover_done	인계 Task 모두 Commit된 시각 (없으면 "—")
rejoin_time, first_assign	27.02, Task 271 (27.12)
dead_now	[W4] (그 순간 DEAD 목록)

전체 카운터: start_time · DEAD 수 · REJOIN 수 · TAKEOVER 수 · REJECT_STALE 수 · max_concurrent_dead. CKPT 절약률은 Worker 로그로 계산.

```
# 제안 양식 (Python 예, 언어 자유)
class TaskEntry:                    # task[k], k = 0..999
    state = PENDING; owner = None; epoch = 0; result = None; commit_count = 0; lock = Mutex()
class WorkerEntry:                  # workers[i], i = 1..10 — 전체를 wt_lock 하나로 보호
    state = ALIVE; lastHB = 0.0; rejoin_count = 0; sock = None; send_lock = Mutex(); peer = (ip, port); ring_ok = False
pending = deque(range(1000)); done_count = 0; all_done = Condition(Mutex())
req_q = deque(); req_cv = Condition(Mutex())   # 항목 = (sock, msg), HELLO 전에는 번호를 모름
```

2. Worker (프로세스 10개가 각자 보유)

묶음	데이터	형식 / 초기값	보호 · 비고
설정 · 계산	id, coord 주소, peer_port, step_delay, small_primes	실행 인자 (peer_port = 7000 + i), 소수 1,229개는 시작 시 한 번	읽기 전용
링	buddy (j, ip, port)	RING으로 받음, j = i mod 10 + 1	고정 (끝까지 변경 없음)
★ 상태	state	CONNECTING → RUNNING / CRASHED / REJOINING	state_lock + cv (HB · Compute가 대기)
	★ crash_gen	int, 0 (CRASH마다 +1)	
	stopping	bool, False (BYE 수신 시 True)	
★ 로컬 Task Queue	항목	(k, e, start_step, partial) — ASSIGN이면 (k, 1, 1, 0)	q_lock + q_cv
	★ requested, no_task	bool, False (REQ 대기 / NO_TASK 수신)	
★ Checkpoint Store	store	dict k → (epoch, step, partial), {} · (e, s) 사전식으로 더 크면 덮어씀	store_lock · 삭제 규칙 없음
진행 중 Task	(k, e, s, partial, gen)	Compute Thread 지역 변수	gen = 꺼낸 순간의 crash_gen
연결	coord_sock(+send_lock), buddy_sock, launcher_sock, 선행 Worker의 CKPT 연결	Socket, 각자 recv 버퍼	crash 중에도 닫지 않음
READY	start_received, launcher_connected, ready_sent	bool, False (Control Recv · Peer Server가 각각 설정)	**state_lock 안에서** 둘 다 True · ready_sent 미전송이면 ready_sent = True → Lock 밖에서 READY i
통계	CKPT 송·수신 수, ACK · REJECT 수, crash 횟수, 인계 재개 step	int / list	로그 · CKPT 절약률 계산용

```
# 제안 양식
state = CONNECTING; crash_gen = 0; stopping = False
                        # state_lock + cv
local_q = deque()       # (k, e, start_step, partial)
requested = no_task = False   # q_lock + q_cv
store = {}              # k -> (epoch, step, partial)
                        # store_lock
# CRASH: state = CRASHED; crash_gen += 1
#   local_q.clear(); requested = no_task = False
#   store.clear()
```

Worker에 없는 것: owner · Task의 공식 epoch 이력 · 다른 Worker의 생존 상태 · 합계. epoch는 ASSIGN/TAKEOVER로 받은 값을 그대로 저장해 쓴다.

Lock 순서: q_lock → state_lock → store_lock. TAKEOVER 처리는 q_lock 안에서 store_lock을 잠깐 잡아 Store[k]를 읽는다.

3. Chaos Launcher

묶음	데이터	형식 / 초기값
설정	seed, rng, workers, peer_base, step_delay	rng = Random(seed), 10, 7000, 100 ms
★ Worker 상태 표 [1..10]	status · proc · ctrl_sock	RUNNING / CRASHED / REJOINING / EXITED · 프로세스 핸들 · 제어 연결
	restart_at	CRASH 시각 + D(10~15), 없으면 None
	crash_count	int, 0
★ 스케줄	t0	READY 10개를 받은 시각
	next_crash	t0 + 10~20, 이후 CRASH + 5~15
	blocked	2개 crash 중 대기 여부 (REJOINED 후 I 새로)
	ready_count	0 → 10
★ 사건 기록	events	(#, victim, crash_t, D, restart_t, rejoined_t, down) → Launcher.txt. 각 시각은 **t0 기준 초와 실제 시각 (HH:MM:SS.mmm)을 함께** 기록 (Coordinator 로그와 대조)

```
# 제안 양식
rng = Random(seed)
status = {i: RUNNING
          for i in range(1, 11)}
restart_at = {}   # i -> 시각
crash_count = {}  # i -> int
t0 = next_crash = None
blocked = False
events = []  # dict(no, victim,
  # crash_t, D, restart_t,
  # rejoined_t, down)
```

Launcher에 없는 것: Task · epoch · owner · HB 정보. Coordinator와 통신하지 않으며, Coordinator도 이 스케줄을 모른다.

4. 한눈에 보기 — 어떤 데이터가 어디에 있나

데이터	Coordinator	Worker	Launcher
Task의 owner · epoch · state	★ **원본**	받은 e만 Queue 항목에	—
생존 판정 (ALIVE / SUSPECT / DEAD)	★ (HB 기반)	—	—
CKPT (step, partial)	— (모름)	★ Buddy의 Store	—
crash 여부 (CRASHED)	— (HB로 추정)	★ 자기 state	★ status 표
합계 · commit_count	★	—	—
crash 스케줄 · seed	— (모름)	—	★
지표 (감지 · 복구 · 재합류)	★ crash 기록	CKPT 절약률용 재개 step	CRASH · RESTART 시각 (대조용)

원칙: 기준값(owner · epoch · 생존 상태)은 Coordinator 한 곳에만 두고 Worker는 받은 값을 그대로 쓴다. Lock 순서: Coordinator WT → Task, Worker q → state → store.